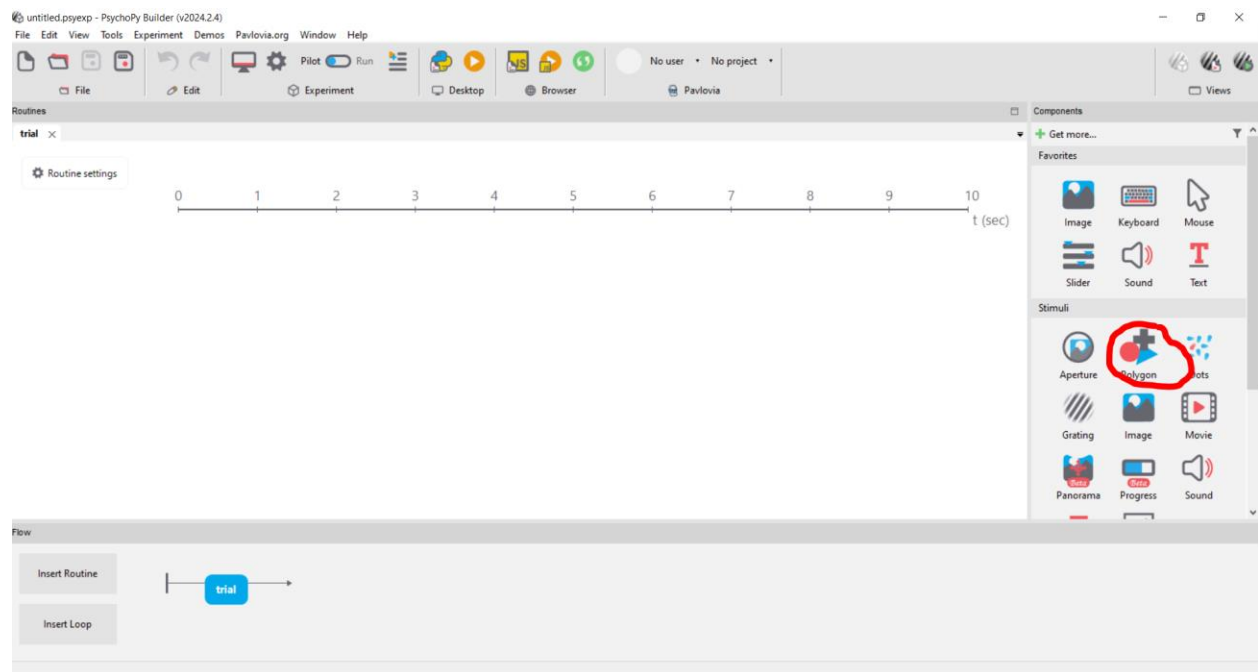


Step-by-Step: a Simple Continuous Psychophysics Experiment Designed in PsychoPy

The following is a step-by-step guide to create a approachably simplified version of the seminal tracking experiment reported by Bonnen et al. (2015) in PsychoPy.

1. Create object to be tracked

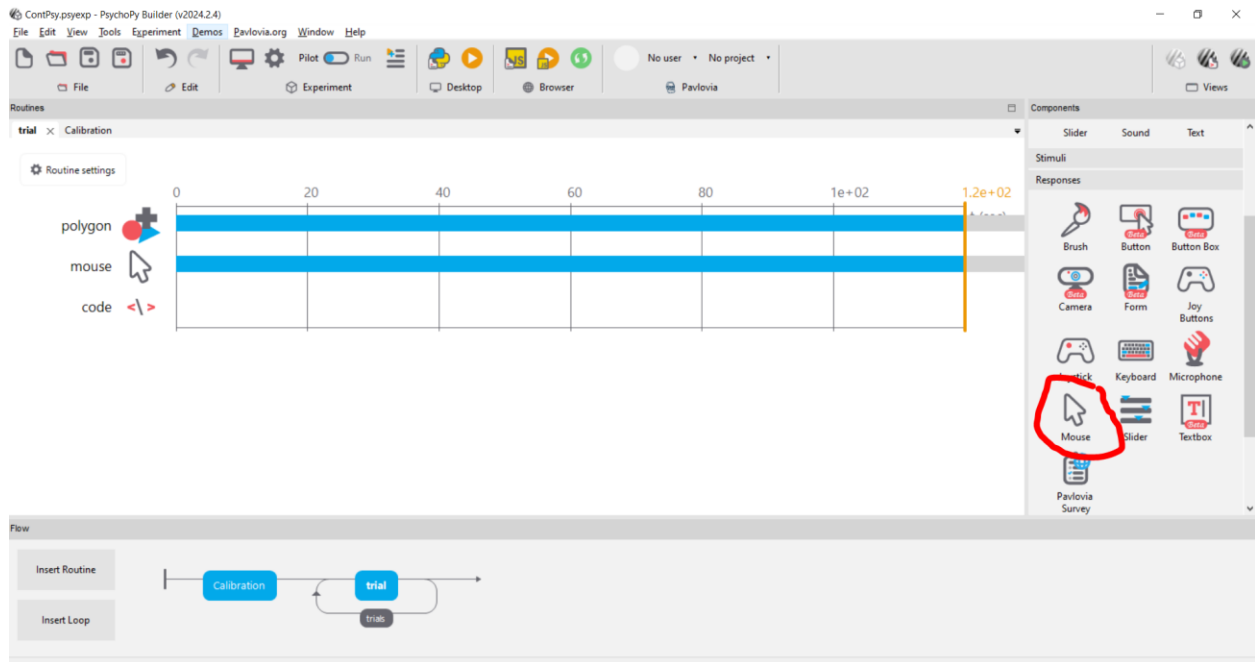
Choose “Polygon” from the right-hand column under “Stimulus”.



- a. Under “Basic”:
 - i. Leave “Duration” empty
- b. Under “Layout”:
 - i. Set Spatial Units to “cm”
- c. Under “Appearance”:
 - i. Write “opacity” in the Opacity field

2. Create a representation of mouse cursor

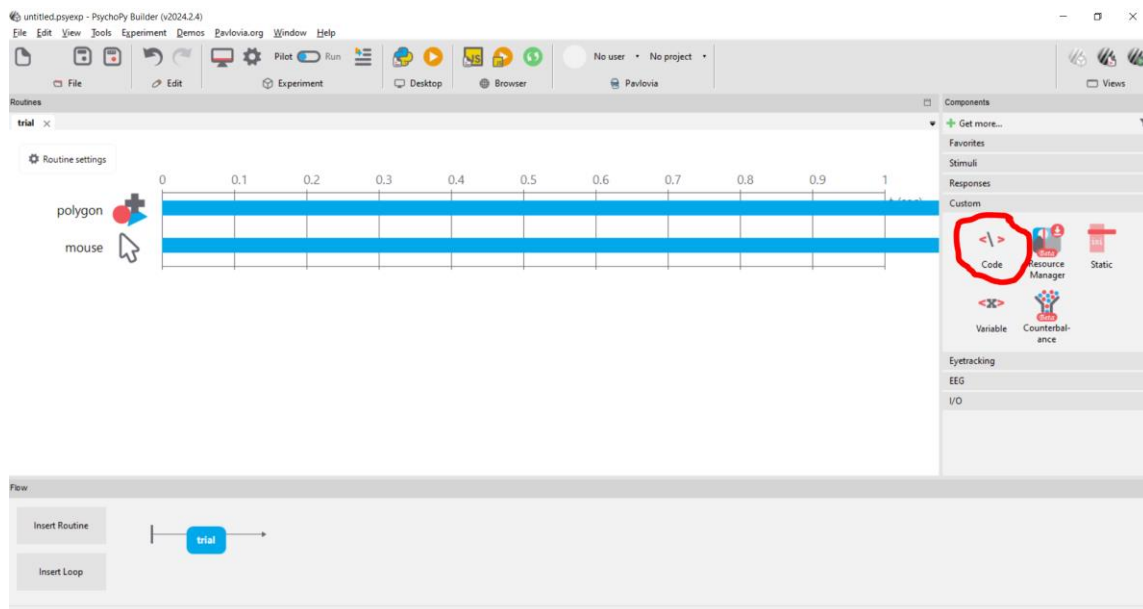
Chose “Mouse” under “Responses”.



a. Under “Basic”:

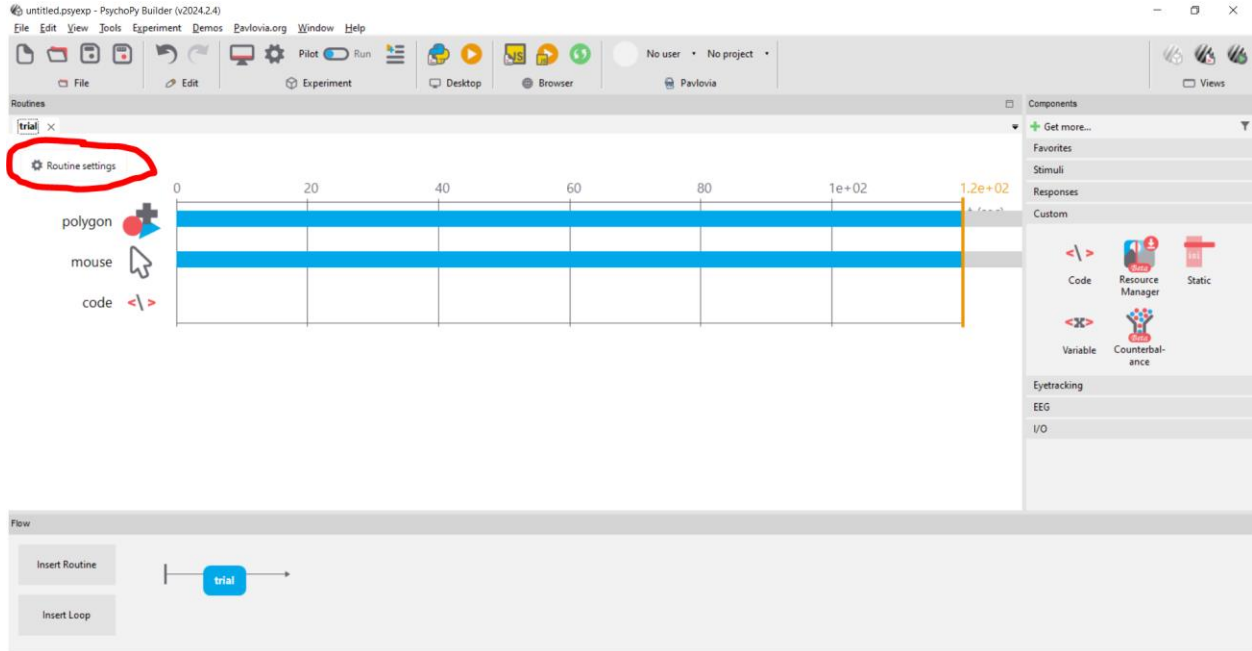
- i. Leave “Duration” empty
- ii. For “End Routine on Press”, choose “never”

3. Create the code object



- a. We are returning to this later!

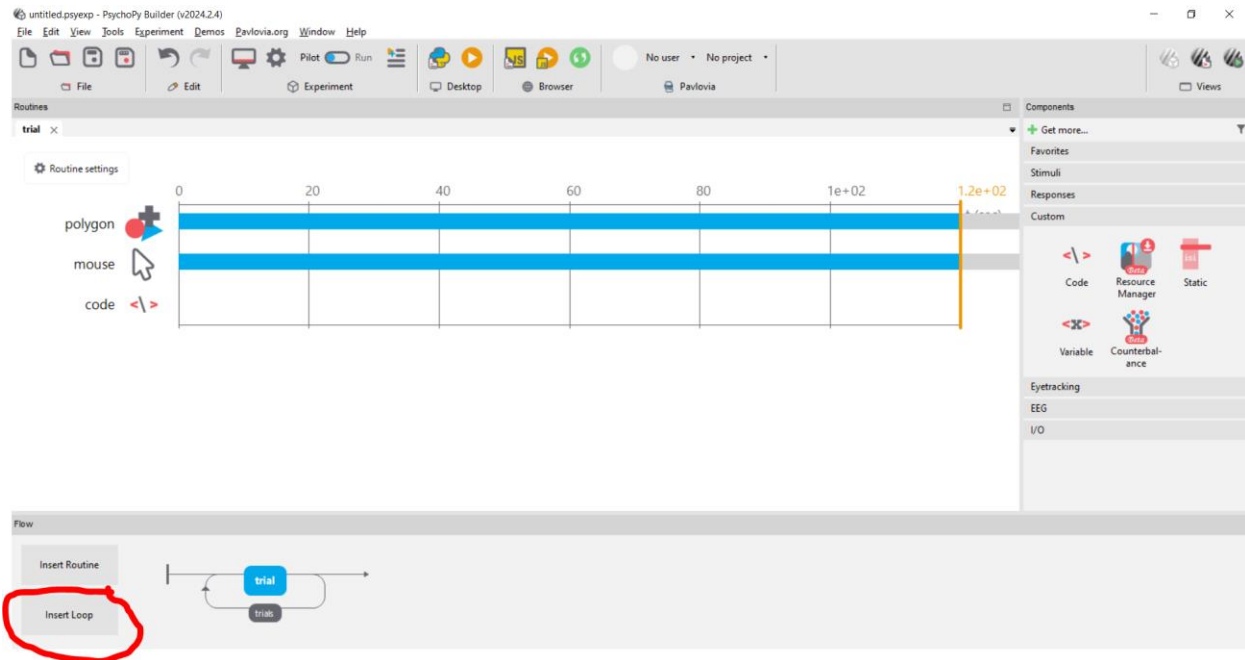
4. Configure Routine



- a. Under “Flow”:
 - i. Set Duration to 120s

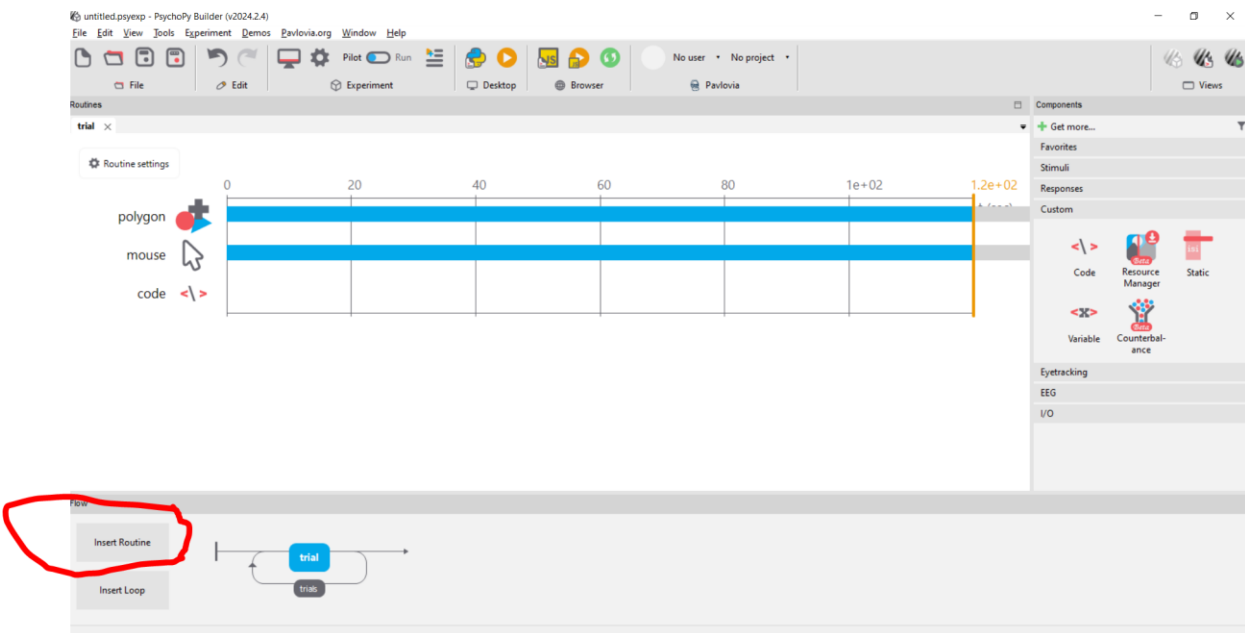
5. Create a loop

(This allows us to run the same routine twice – once with high opacity and once with low opacity. The opacity will be set using the “custom code” feature – see below.)



- a. Click on “Insert Loop” and then click on one of the two dots that appears to the left and right of the “trial” box in the bottom ribbon.
 - i. Set “Num. Repeats” to 2

6. Add calibration routine in the beginning of the experiment



- a. Insert a new Routine before “trial”. Name it “Calibration” and click on the circle to the left of “trial” to insert it.

b. Add a Polygon

i. Under “Basic”:

1. Leave “Duration” empty

ii. Under “Layout”:

1. Set “Size” to (0.05, 0.05)

2. Set “Position” to (0.5, 0.5)

3. Set “Spatial Units” to “cm”

c. Add representation of mouse cursor

i. Under “Basic”:

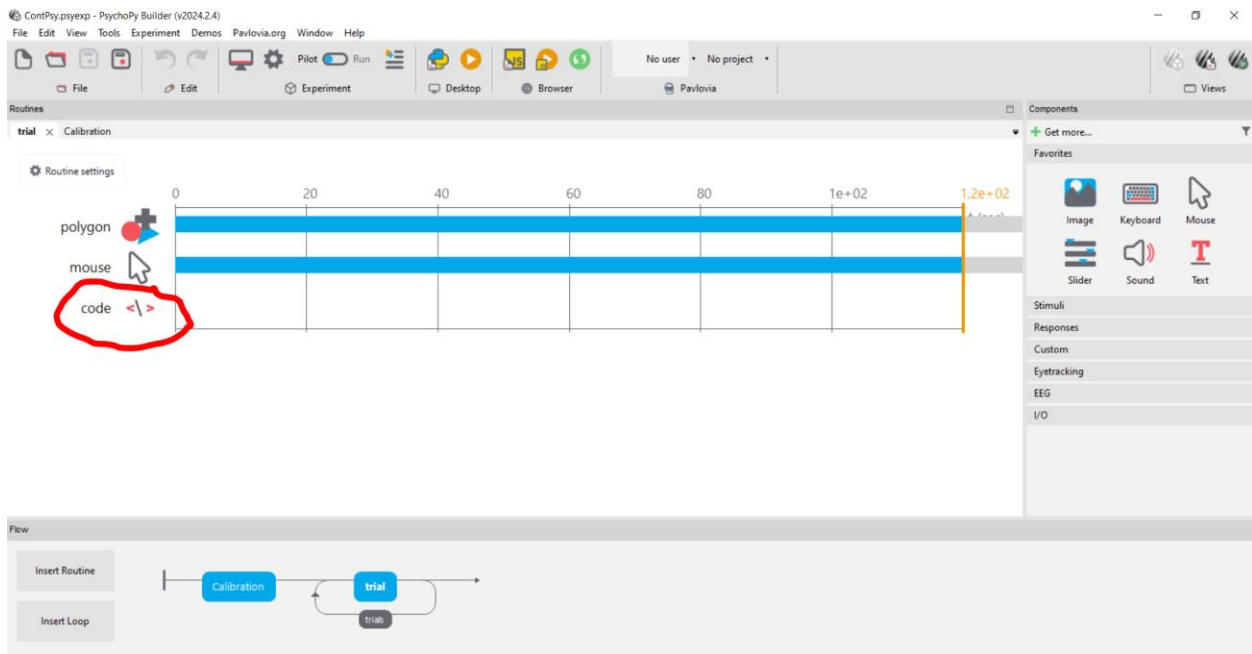
1. Leave “Duration” empty

2. Set “End Routine on press” to “any click”

3. Set “Clickable Stimulus” to “Polygon_2” (the polygon we created for this routine)

7. Fill in code

We use custom code for features that PsychoPy’s graphical interface does not currently support (random walk in stimulus position, saving data frame-by-frame).



a. Under “Before experiment”, add:

```
#Load required packages: numpy for the random walk, time to keep track of the temporal  
#aspect of the experiment  
import numpy
```

```
import time
```

b. Under “Begin Experiment”, add:

```
#Here we randomize which opacity we start with.  
a = [0.03,1]  
opacity = numpy.random.choice(a)
```

c. Under “Begin Routine”, add:

```
#We reset the initial position of the dot at the beginning of each run/routine.  
posx = 0  
posy = 0  
pos = (posx, posy)
```

```
#Save the time in the beginning of each run so we know how long it's been running  
start = time.time()
```

d. Under “Each Frame”, add:

```
#start moving the dot only after 3 seconds  
if time.time() - start > 3:
```

```
#draw the next step for the random walk for x and y directions  
Addx = numpy.random.normal(loc=0,scale = 0.3)  
Addy = numpy.random.normal(loc=0,scale = 0.3)
```

```
#add the next step to the current position,  
#but make sure we don't go off screen (set at +- 6cm from middle of screen)  
if posx + Addx < 6 and posx + Addx > -6:  
    posx = posx + Addx
```

```
if posy + Addy < 6 and posy + Addy > -6:  
    posy = posy + Addy
```

```
pos = (posx, posy)
```

```
#save the relevant data on each frame  
thisExp.addData("time_in_run", time.time() - start)  
thisExp.addData("x_coord_mouse", mouse.getPos()[0])  
thisExp.addData("y_coord_mouse", mouse.getPos()[1])  
thisExp.addData("x_coord_target", posx)  
thisExp.addData("y_coord_target", posy)  
thisExp.addData("opacity", opacity)  
thisExp.nextEntry()
```

e. Under “End Routine”, add:

```
#change opacity from the one we ran first to the other one
if opacity == 0.03:
    opacity = 1
else:
    opacity = 0.03
```